



Road Reorganization (roads)

Recientemente, el Ejército Romano ha adquirido nuevos carruajes, que pueden llevar un número mayor de personas. Desafortunadamente, esto también significa que los carruajes son más grandes y a veces se quedan atascados cuando hay tráfico en dirección opuesta. Con el fin de prevenir esto, el Emperador decide convertir toda carretera del Imperio a carreteras unidireccionales.

Sin embargo, al Emperador le gusta ir a su ciudad natal a menudo, y quiere asegurarse de que cuando esté allí, pueda volver a Roma rápido en caso de que algo importante surja. Para conseguirlo, se quiere asegurar de que el camino más corto de su ciudad natal a Roma no se vuelve más largo de lo que ya es. Adicionalmente, también quiere asegurarse de que puede volver a su ciudad natal, pero en este caso no hace falta que sea lo más rápido posible.

El Imperio Romano consta de N ciudades y M carreteras bidireccionales. La ciudad natal del Emperador es la ciudad 0 y Roma es la ciudad $N - 1$. Formalmente, tiene que elegir un sentido para cada carretera. Tiene que asegurarse que el camino más corto de la ciudad 0 a la ciudad $N - 1$ no se vuelve más largo de lo que ya era antes de que las carreteras se volvieran unidireccionales. Adicionalmente, también se quiere asegurar que existe un camino de vuelta de la ciudad $N - 1$ a la ciudad 0. Tu tarea es orientar las carreteras de esta forma, o informar al Emperador de que no es posible.

⇒ Se garantiza que cualquier ciudad del Imperio Romano se puede alcanzar desde cualquier otra ciudad. Ten en cuenta que este no tiene por qué ser el caso después de orientar las carreteras.

Ten en cuenta que orientar las carreteras de tal forma que podemos ir de 0 a $N - 1$ y volver, sin necesariamente minimizar la longitud de ninguno de los viajes, va a darte puntos igualmente.

Implementación

Vas a tener que enviar un único fichero `.cpp` fuente.



Entre los ficheros adjuntos a esta tarea, vas a encontrar una plantilla `roads.cpp` con un ejemplo de implementación.

Vas a tener que implementar la siguiente función:

C++

```
vector<pair<int, int>> orient_roads(int N, int M, vector<int> A,
vector<int> B, vector<int> W)
```

- El entero N representa el número de ciudades.
- El entero M representa el número de carreteras.
- Los vectores A , B y W describen las carreteras del Imperio Romano: Hay una carretera entre la ciudad $A[i]$ y la ciudad $B[i]$ de longitud $W[i]$.
- Si hay alguna manera de orientar las carreteras de la forma pedida, la función debe retornar un vector de parejas de longitud M . Aquí, la i -ésima pareja $\{F, T\}$ significa que hay una carretera orientada de F a T .
- Si no hay ninguna manera de orientar las carreteras, la función debe devolver un vector vacío.

Se garantiza que hay como máximo una carretera entre dos ciudades y que para todo i se cumple $A[i] \neq B[i]$. Si la función no retorna un vector de longitud 0 o M , se va a considerar la respuesta incorrecta. Todas las carreteras mencionadas en la entrada deben estar presentes en la salida exactamente una vez y ninguna otra carretera debe estar presente. El orden de las carreteras en el vector de salida no es importante.

Sample Grader

Entre los ficheros adjuntos a esta tarea vas a encontrar una versión simplificada del juez usada durante la evaluación, que puedes usar para comprobar tus soluciones localmente. El juez de muestra lee de `stdin`, llama la función `find_factions` y escribe en `stdout` usando el siguiente formato.

La entrada consta de $M + 1$ líneas, conteniendo:

- Línea 1: los enteros N y M .
- Línea $2 + i$ ($0 \leq i < M$): los enteros A_i , B_i y W_i .

La salida consta de 1 o $M + 1$ líneas, conteniendo:

- Línea 1: **Yes** o **No**, indicando si es posible orientar las carreteras de la forma pedida.
- Si la respuesta es **Yes**: Línea $2 + i$ ($0 \leq i < M$): F_i y T_i , indicando que la carretera entre estas dos ciudades debe estar orientada de F_i a T_i .

En la salida, las orientaciones de las carreteras son impresas en el mismo orden en el que son devueltas en la función.

Restricciones

- $3 \leq N \leq 100\,000$.
- $N - 1 \leq M \leq 100\,000$.
- $0 \leq A_i, B_i \leq N - 1$ y $A_i \neq B_i$.
- $1 \leq W_i \leq 1\,000\,000\,000$.
- Se garantiza que toda ciudad del Imperio Romano se puede alcanzar desde cualquier otra ciudad.

Puntuación

Tu programa va a ser comprobado contra diferentes casos de prueba agrupados en subtareas. Para conseguir la puntuación completa de una subtask, tu programa debe resolver todos sus casos de prueba. Si la solución es válida pero no óptima, vas a obtener la mitad de los puntos de la subtask.

- **Subtask 0 [0 puntos]**: caso de prueba de muestra.
- **Subtask 1 [6 puntos]**: $\max(N, M) \leq 20$.
- **Subtask 2 [11 puntos]**: $M \leq N$.
- **Subtask 3 [17 puntos]**: Se garantiza que hay un camino directo entre la ciudad 0 y la ciudad $N - 1$.
- **Subtask 4 [18 puntos]**: $\max(N, M) \leq 2000$.
- **Subtask 5 [17 puntos]**: $W_i = 1$.
- **Subtask 6 [31 puntos]**: Sin restricciones adicionales.

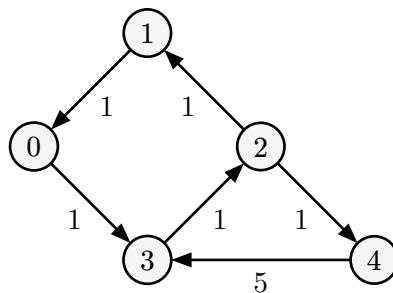


Puedes obtener la mitad de los puntos de una subtask si el camino más corto entre la ciudad 0 a la $N - 1$ después de orientar las carreteras no es de la misma longitud que el camino más corto entre ellas antes de orientarlas, incluso si no hay forma de orientar las carreteras de forma que se pueda satisfacer al Emperador.

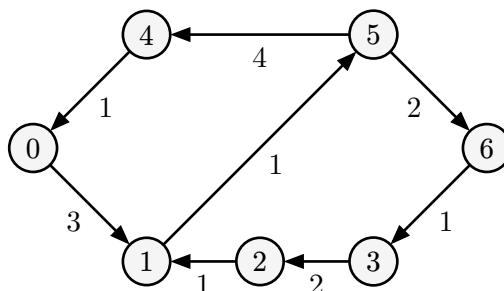
Ejemplos de entrada/salida

stdin	stdout
5 6 0 1 1 1 2 1 0 3 1 3 2 1 2 4 1 3 4 5	Yes 1 0 2 1 0 3 3 2 2 4 4 3
7 8 0 1 3 1 2 1 2 3 2 3 6 1 0 4 1 4 5 4 5 6 2 1 5 1	Yes 0 1 2 1 3 2 6 3 4 0 5 4 5 6 1 5
8 9 0 1 3 1 2 4 2 4 2 0 3 6 3 4 1 4 5 3 5 6 1 6 7 2 5 7 3	No

Explicación



En el primer ejemplo, podemos ir de la ciudad 0 a la cuatro 4 via $0 \rightarrow 3 \rightarrow 2 \rightarrow 4$. Este camino tiene longitud 3, que también es la longitud del camino más corto antes de orientar las carreteras. Podemos volver con el camino $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$. No importa lo largo que sea este camino.



En el segundo ejemplo, podemos ir de 0 a 6 con el camino $0 \rightarrow 1 \rightarrow 5 \rightarrow 6$ y podemos volver usando $6 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 5 \rightarrow 4 \rightarrow 0$.

En el tercer ejemplo, se puede demostrar que ninguna orientación de carreteras puede hacer feliz al Emperador.